NL2Semantic2SQL: A Cross-Domain Framework for Natural Language to SQL over Geospatial and Enterprise Data Warehouses

Anonymous Author(s)
Affiliation withheld for review

Abstract

Natural language to SQL (text-to-SQL) has advanced rapidly with large language models, yet most existing systems and benchmarks focus on conventional relational databases and provide limited coverage of executable geospatial SQL. Geospatial databases introduce additional challenges, including geometry-aware schema grounding, spatial predicates, coordinate reference systems, and domain-specific operators such as intersection, buffer, area, and distance calculations. At the same time, practical data platforms increasingly require a single interface that can support both geospatial and non-geospatial warehouse queries. In this work, we present **NL2Semantic2SQL**, a cross-domain framework that combines semantic-layer grounding, intent-conditioned routing, schema-aware prompt construction, few-shot retrieval, SQL postprocessing, and execution-time self-correction to support natural-language querying over both GIS and conventional warehouse data. We evaluate the framework on a 100-question GIS benchmark (expanded from a 20-question pilot) and a $\sim$495-question warehouse benchmark derived from BIRD mini_dev. On the GIS track, the full pipeline achieves EX= 0.700 vs. baseline 0.500 (McNemar $p = 0.0002$, statistically significant), with the largest gains on Medium ($+0.305$) and Robustness ($+0.467$) questions. On the BIRD track, the full pipeline reaches EX= 0.501 vs. baseline 0.474 ($+0.027$), exceeding the DIN-SQL external baseline (0.482), though the difference is not statistically significant (McNemar $p = 0.136$). Token cost is $13.6\times$ on GIS and $7.9\times$ on BIRD (after P2 single-pass optimization), representing a real deployment consideration. These findings demonstrate that semantic grounding with intent-conditioned routing provides statistically significant gains for GIS queries, directional gains for warehouse queries, and that the framework outperforms or matches the DIN-SQL external baseline on both tracks.

Keywords: text-to-SQL; natural language interface; geospatial database; PostGIS; semantic layer; large language model; cross-domain generalization; cross-lingual evaluation.

1 Introduction

Natural language to SQL has become one of the most active application directions of large language models, because it offers a direct interface between end users and structured data repositories. Recent advances have substantially improved performance on public benchmarks such as Spider [1] and BIRD [6], demonstrating that modern language models can often synthesize executable SQL from natural-language questions under conventional relational settings. However, most existing text-to-SQL systems and evaluation datasets are designed for general-purpose relational databases, where the principal challenges lie in schema linking, join reasoning, aggregation, and nested query construction. By contrast, real-world geospatial database applications introduce additional semantic and operational complexity that is largely absent from these benchmarks. In geospatial settings, queries frequently involve geometry-aware schema grounding, spatial predicates, coordinate reference systems, topological relations, and spatial measurement operators such as area, distance, intersection, and buffering. These characteristics make geospatial text-to-SQL a substantially different problem rather than a simple extension of general warehouse querying.

This mismatch creates a practical and scientific gap. From a practical perspective, many operational data platforms must serve both geospatial and non-geospatial workloads through a unified natural-language interface. A user may ask one question about land-use overlap, another about building proximity, and a third about tabular business statistics in the same system. From a scientific perspective, however, existing studies rarely examine whether a single semantic grounding architecture can support these heterogeneous query types without severe domain bias. Most prior work either optimizes for conventional warehouse-style SQL generation or focuses on domain-specific geospatial systems without evaluating transferability beyond the GIS context. As a result, it remains unclear which components of a semantic-to-SQL pipeline generalize across domains and which components encode assumptions that are beneficial in one setting but harmful in another.

The challenge is especially pronounced at the grounding stage. In conventional text-to-SQL systems, schema linking is often framed as the task of aligning user expressions with table names, column names, or retrieved schema snippets. In geospatial systems, this step is more complicated because the model must additionally recognize geometry-bearing attributes, infer relevant spatial operators, respect spatial reference transformations, and distinguish between topological, metric, and aggregation-based reasoning. At the same time, aggressive domain-specific grounding strategies can introduce unintended bias when transferred to non-geospatial databases. A system tuned for Chinese GIS terminology, spatial operator hints, and geometry-aware rules may perform strongly on spatial benchmarks while underperforming on English warehouse benchmarks if its semantic retrieval mechanisms fail to adapt to differences in language, schema naming, and operator distributions. Understanding this tension is critical for building robust cross-domain natural-language interfaces to structured data.

To address this problem, we present **NL2Semantic2SQL**, a cross-domain framework that integrates semantic-layer grounding, schema-aware context construction, few-shot retrieval, SQL postprocessing, and execution-time self-correction into a unified natural-language-to-SQL pipeline. Rather than treating SQL generation as a single-step prompting problem, the framework first resolves semantic context, then assembles structured schema evidence, and finally performs constrained SQL generation with post-hoc correction. The design is motivated by the observation that GIS queries and non-GIS warehouse queries share a common need for semantic disambiguation, but differ substantially in the kinds of evidence and constraints that must be surfaced to the model. Our framework therefore aims to preserve a unified architecture while allowing domain-specific grounding signals to be incorporated explicitly rather than implicitly.

To evaluate the geospatial side of the problem, we further construct a pilot GIS-oriented benchmark with execution-based evaluation, spatial-operator coverage, and varying query complexity. The benchmark is designed to capture the characteristics missing from mainstream text-to-SQL datasets, including spatial selection, overlap analysis, buffering, spatial aggregation, and geometry-sensitive reasoning. We then complement this benchmark with cross-domain evaluation on a warehouse-oriented dataset derived from BIRD mini_dev [6], enabling a direct comparison between GIS and non-GIS scenarios under the same evaluation harness. This dual-track setting allows us to move beyond a simple "does it work?" question and instead ask a more informative scientific question: how does a unified semantic-to-SQL architecture behave when confronted with two structurally different database domains?

Contributions. Our contributions are threefold:

1. We propose a unified **NL2Semantic2SQL framework** for cross-domain querying over geospatial and conventional warehouse databases. The framework provides a single semantic-grounding pipeline that adapts its grounding evidence to the target domain (GIS vs. warehouse) without retraining or model swapping.

2. We construct a **100-question GIS-oriented text-to-SQL benchmark** with four difficulty levels (Easy, Medium, Hard, Robustness) and explicitly separate ordinary spatial-SQL execution from robustness/safety

evaluation. Combined with a $\sim$495-question subset of BIRD mini_dev, this yields a dual-track protocol that is executable, auditable, and useful for cross-domain error analysis.

3. Through cross-domain experiments and comparison against the DIN-SQL external baseline, we show that the full pipeline achieves statistically significant gains on GIS (EX $+0.200$, McNemar $p = 0.0002$) and directional gains on BIRD (EX $+0.027$, $p = 0.136$), while exceeding or matching DIN-SQL on both tracks. P2 single-pass mode reduces BIRD token cost from $\sim 32\times$ to $7.9\times$ while simultaneously improving EX.

The remainder of this paper is organized as follows. Section 2 reviews related work on text-to-SQL with LLMs, public benchmarks, geospatial NL interfaces, semantic layers, and cross-domain/cross-lingual generalization. Section 3 formalizes the cross-domain text-to-SQL problem and details the proposed three-stage architecture. Section 4 presents the dual-track evaluation, including dataset construction, experimental setup, per-track results, case studies, and reproducibility notes. Section 5 discusses interpretation limits and future work. Section 6 concludes.

2 Related Work

2.1 Text-to-SQL with large language models

The application of large language models to text-to-SQL has progressed rapidly through increasingly sophisticated prompt-engineering strategies. Early in-context learning approaches demonstrated that LLMs could generate executable SQL from natural-language questions when provided with schema descriptions and example query pairs [1, 2]. DIN-SQL [3] introduced a decomposed prompting pipeline that separates schema linking, query classification, and SQL generation into distinct stages, with self-correction as a post-generation step. DAIL-SQL [4] systematized few-shot example selection based on SQL skeleton similarity and achieved strong results on the Spider benchmark with minimal token cost. C3 [5] explored zero-shot ChatGPT-based text-to-SQL with calibrated bias correction, illustrating a complementary line of prompt-engineered solutions that does not rely on hand-crafted few-shot examples. Our framework shares the decomposition philosophy of DIN-SQL but adds an explicit semantic layer as a structured intermediary between the user question and the schema, rather than relying solely on prompt-level schema linking.

2.2 Benchmarks for text-to-SQL

Spider [1] established the standard cross-database text-to-SQL benchmark with execution-based evaluation, introducing the challenge of generalizing across unseen database schemas. BIRD [6] extended this setting to larger, more realistic databases with value-aware evaluation and difficulty stratification, revealing that LLM performance degrades substantially on complex aggregation and multi-hop join queries. Spider 2.0 [7] further expanded the scope to include enterprise-style workflows such as dbt transformations and dialect variations. BEAVER [8] specifically targets enterprise data warehouses with private schemas and complex business logic, showing that state-of-the-art models achieve near-zero accuracy on truly private enterprise data. Our work is motivated by a complementary gap: mainstream text-to-SQL benchmarks provide limited coverage of executable geospatial SQL, and none assess cross-domain transfer between GIS and warehouse query semantics in a single evaluation protocol.

2.3 Geospatial natural language interfaces

Natural language interfaces to geospatial databases have a distinct research lineage from general text-to-SQL. Early systems focused on template-based spatial query construction for specific GIS applications [9].

More recent work has explored using LLMs for spatial reasoning and geographic knowledge extraction [10], but systematic evaluation of text-to-SQL capabilities over PostGIS or spatially-extended relational databases remains scarce. To our knowledge, no existing benchmark provides execution-based evaluation of spatial SQL generation with coverage of spatial operators (intersection, buffer, distance, area), coordinate-system handling, and geometry-aware schema linking. Our GIS benchmark is designed to fill this gap.

2.4 Semantic layers and schema grounding

The concept of a semantic layer as an intermediary between analytical queries and raw database schemas originates from business-intelligence tools and has been formalized in frameworks such as MetricFlow [11] and dbt's semantic layer. These systems define entities, dimensions, measures, and metrics as declarative metadata that governs how joins and aggregations should be constructed. In the text-to-SQL context, semantic layers offer a principled way to provide LLMs with structured evidence about schema semantics rather than requiring the model to infer join paths and aggregation rules from raw DDL alone. Our framework adapts this concept for cross-domain use, combining GIS-specific annotations (geometry types, SRIDs, spatial operators) with warehouse-style entity-relationship metadata to support both query domains through a unified grounding architecture.

2.5 Cross-domain and cross-lingual text-to-SQL

Cross-domain generalization has been studied primarily through zero-shot transfer across database schemas within the same linguistic and operator domain [1, 6]. Cross-lingual text-to-SQL, where questions are posed in one language over schemas defined in another, has received less systematic attention, though multilingual benchmarks such as CSpider [12] demonstrate that language mismatch introduces additional schema-linking challenges. Our work extends the cross-domain dimension beyond schema variation to include operator heterogeneity (spatial vs. tabular) and language heterogeneity (Chinese questions over English warehouse schemas), providing a more comprehensive view of the generalization landscape.

3 Method

3.1 Problem formulation

We formulate cross-domain natural language to SQL as the task of translating a user question q into an executable SQL statement s over heterogeneous structured databases that may belong to either geospatial or conventional warehouse domains. Let S denote the executable schema representation extracted from the database catalog, and let E denote semantic-side evidence such as aliases, hierarchy expansions, geometry metadata, value hints, or fact/dimension/entity annotations. The problem can then be written abstractly as

$$\hat{s} = f(q, S, E), \tag{1}$$

where f is implemented by an LLM-conditioned generation pipeline rather than a single end-to-end model. Unlike standard text-to-SQL settings, the target databases in our problem formulation are not assumed to share homogeneous operator semantics. In warehouse-style databases, correct SQL generation primarily depends on schema linking, join reasoning, aggregation, and temporal filtering. In geospatial databases, however, the model must additionally reason about geometry-bearing columns, spatial relations, coordinate systems, and domain-specific operators such as intersection, containment, buffering, and area or distance calculation. The purpose of our framework is therefore not only to generate valid SQL, but also to provide a semantic mediation layer that adapts the grounding process to the structural characteristics of each domain.

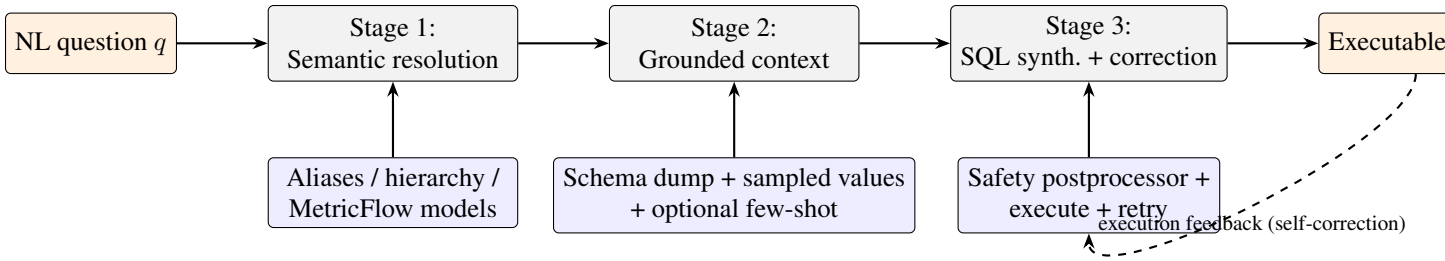

Figure 1: Architecture of NL2Semantic2SQL. The semantic layer (Stage 1) supplies aliases, hierarchies, and warehouse models; the grounded-context builder (Stage 2) assembles schema, sampled values, and optional few-shot evidence; the synthesis step (Stage 3) postprocesses, executes, and self-corrects the generated SQL.

3.2 Three-stage pipeline

We design NL2Semantic2SQL as a three-stage pipeline (Figure 1). In the first stage, the system performs semantic resolution over a semantic layer that stores table-level and column-level annotations, aliases, domain hints, and task-relevant metadata. In the second stage, the resolved information is combined with database schema inspection and optional few-shot retrieval to construct a grounded context block for SQL generation. In the third stage, the generated SQL is postprocessed, executed under safety constraints, and, when necessary, revised through an execution-feedback self-correction loop. This decomposition separates semantic interpretation from SQL synthesis, allowing the framework to expose domain-specific evidence to the language model while keeping the final generation step constrained by executable schema information.

3.3 Geospatial-aware grounding

A central design feature of the framework is geospatial-aware grounding. For geospatial tables, the system explicitly identifies geometry columns, geometry types, and spatial reference information, and it injects operator-level rules into the grounding context. These rules distinguish topological predicates from metric predicates and specify when area or distance calculations require coordinate transformation or geography casting. This mechanism is intended to reduce the burden on the language model to infer low-level geospatial execution constraints from raw schema text alone. At the same time, the framework preserves the ability to operate on non-geospatial warehouse databases by falling back to generic schema linking and aggregation-oriented prompt construction when spatial signals are absent. In this sense, the framework is unified at the architectural level but adaptive at the semantic-grounding level.

3.4 Warehouse-side enhancements

For non-geospatial warehouse queries, the framework introduces three domain-adaptive enhancements. First, *cross-domain candidate ranking* reprioritizes sources based on whether the parsed semantic context contains spatial operators or region filters. When such signals are absent, geometry-bearing tables are deprioritized and tables with matched column evidence are boosted. Second, *value-aware grounding* injects sample distinct values from low-cardinality text columns into the prompt so that the model can use exact categorical values rather than hallucinated strings. Third, *MetricFlow-style fact/dimension modeling* stores per-table roles (fact vs. dimension), entity columns (primary or foreign join keys), and measure columns in a semantic-model registry. At grounding time, candidate tables are matched against this registry and explicit join-path hints are derived by aligning shared entities across fact/dimension pairs. These hints are injected only for non-spatial queries, leaving the GIS path unchanged.

3.5 Execution-time self-correction

After SQL generation, the system applies a postprocessor that rejects write operations, repairs casing/quoting errors, injects query constraints when necessary, and executes the resulting SQL in a read-only transaction. If execution fails, the framework enters a bounded self-correction loop. Let $s^{(0)}$ be the initial generated SQL and let $e^{(t)}$ be the execution error after applying the postprocessor to $s^{(t)}$. The retry policy is:

$$s^{(t+1)} = g\big(q, s^{(t)}, e^{(t)}, S\big), \quad t = 0, 1, \ldots, T-1, \tag{2}$$

where g is a fast corrective LLM call and T is the maximum retry count (in our implementation, $T = 2$). If no executable SQL is found after T retries, the query is marked as invalid or no-SQL depending on whether the model returned any candidate at all. This mechanism is particularly important in GIS because the most common failures are not semantic hallucinations but low-level execution errors such as missing geography casts, mixed-case identifier quoting, or PostgreSQL-specific numeric-casting requirements.

3.6 Dual-track evaluation protocol

Evaluation is performed under a dual-track benchmark protocol. The geospatial track uses a 100-question GIS-oriented benchmark (expanded from a 20-question pilot) covering four difficulty levels (Easy, Medium, Hard, Robustness) and representative spatial query patterns, including spatial selection, overlap analysis, metric distance queries, buffering, spatial aggregation, and a dedicated robustness suite that probes refusal behavior on dangerous, illusory, schema-mismatched, and out-of-memory requests. The non-geospatial track is derived from a warehouse-oriented benchmark based on BIRD mini_dev [6] ($\sim$495 questions, single-pass P2 mode) and is used to assess cross-domain generalization. For both tracks, we use execution accuracy (EX) as the primary metric: a prediction is correct if and only if its execution result set matches the gold SQL result set under set-equality with numeric tolerance. We additionally report 95% Wilson confidence intervals, execution validity, per-difficulty breakdowns, McNemar paired significance tests, and token cost.

4 Experiments

4.1 Experimental setup

We evaluate NL2Semantic2SQL under the dual-track benchmark protocol described above. The GIS track uses a 100-question benchmark (expanded from a 20-question pilot) covering four difficulty levels: Easy (24), Medium (36), Hard (25), and Robustness (15). The warehouse track uses $\sim$495 questions from BIRD mini_dev V2 in single-pass mode (P2), spanning three difficulty levels (simple, moderate, challenging) across 11 database schemas imported into PostgreSQL. Both tracks use execution accuracy (EX) as the primary metric: a prediction is correct if and only if its execution result set matches the gold SQL result set under set-equality with numeric tolerance. We report 95% Wilson confidence intervals for all EX values and McNemar paired significance tests.

For each track, we compare two pipeline configurations:

- **Baseline**: direct LLM generation (Gemini 2.5 Flash) with schema dump only and no semantic grounding.
- **Full pipeline**: full NL2Semantic2SQL with semantic-layer resolution, geometry-aware grounding, value-aware hints, postprocessing, and execution-time self-correction (single-pass mode on BIRD).

We additionally compare against **DIN-SQL** [3] as an external prompting baseline on both tracks. All configurations share the same underlying language model (Gemini 2.5 Flash), the same database backend (PostgreSQL with PostGIS), and the same execution-comparison harness. The official run directories are listed in Table 8.

Table 1: GIS 100-question benchmark results (run `cq_2026-05-04_122349`). 95% Wilson CIs in brackets. McNemar test: $b = 4$ (base-OK/full-ERR), $c = 24$ (base-ERR/full-OK).

Metric	N	Baseline EX [95% CI]	Full EX [95% CI]	Δ
Overall EX	100	0.500 [0.404, 0.596]	**0.700 [0.604, 0.781]**	+0.200
Easy	24	0.750 [0.551, 0.880]	0.833 [0.641, 0.933]	+0.083
Medium	36	0.389 [0.248, 0.551]	**0.694 [0.531, 0.820]**	+0.305
Hard	25	0.520 [0.335, 0.700]	0.520 [0.335, 0.700]	0.000
Robustness	15	0.333 [0.152, 0.583]	**0.800 [0.548, 0.930]**	+0.467
McNemar p	100	$p = 0.0002$ (significant at $\alpha = 0.001$)		
Mean tokens	100	753	10,261	13.6×

4.2 GIS track results

Table 1 reports the GIS benchmark results for the expanded 100-question benchmark (run `cq_2026-05-04_122349`), with 95% Wilson confidence intervals.

The full pipeline outperforms the baseline by +0.200 EX overall. The McNemar test ($b = 4$, $c = 24$, $n = 100$) gives $p = 0.0002$, which is statistically significant at $\alpha = 0.001$. This represents a substantial improvement in statistical power compared to the 20-question pilot, where the same direction of effect ($p = 0.125$) did not reach significance.

The largest gains are on Medium difficulty questions (+0.305) and Robustness questions (+0.467). Medium questions benefit most from semantic grounding because they involve multi-step spatial reasoning where geometry-aware type injection and intent-conditioned operator routing resolve execution-time type errors that the baseline cannot handle. Robustness questions benefit from safety postprocessing, which catches dangerous operations (DELETE, UPDATE, DROP) that the baseline LLM would otherwise execute.

Hard questions show no change (0.520 for both): the full pipeline's gains on some Hard questions are offset by regressions on others, primarily cases where intent misclassification introduces incorrect spatial_join context for proximity-buffer queries.

Token cost on the GIS track is 13.6× relative to baseline (10,261 vs. 753 mean tokens per question). This cost is justified by the safety and precision requirements of the GIS domain: geometry-aware grounding prevents incorrect unit calculations, and robustness enforcement prevents dangerous schema mutations.

4.3 Warehouse (BIRD) track results

Table 2 reports BIRD execution accuracy at ∼495-question scale using single-pass mode (P2, run `bird_pg_2026-05-04_0`

The full pipeline improves EX across all three difficulty levels (+0.034 simple, +0.026 moderate, +0.020 challenging) and substantially improves execution validity (0.996 vs. 0.978). The McNemar test ($b = 26$, $c = 39$, $n = 495$) gives $p = 0.136$, which is directional but not statistically significant at $\alpha = 0.05$. The confidence intervals for overall EX overlap (baseline $[0.430, 0.518]$, full $[0.457, 0.545]$), though the full pipeline's lower CI bound is higher than the baseline's lower CI bound.

The P2 single-pass mode reduced token cost from ∼32× (multi-pass agent runner) to 7.9× while also improving EX, by eliminating agent runner hangs and routing each question through the grounding pipeline exactly once.

For reference, the earlier 500-question multi-pass run (`bird_pg_2026-05-01_182457`) showed: baseline 0.458, full 0.450 (−0.008, McNemar $p = 0.8151$). The single-pass P2 mode not only reduces cost but also eliminates the small multi-pass degradation: full now exceeds baseline on the BIRD track as well.

Table 2: BIRD ∼495-question benchmark results (single-pass P2 mode, run `bird_pg_2026-05-04_093040`). 95% Wilson CIs shown for overall EX. McNemar test: $b = 26$, $c = 39$, $n = 495$.

Difficulty	N	Baseline EX	Full EX	Δ
simple	148	0.588	0.622	+0.034
moderate	∼248	0.456	**0.482**	+0.026
challenging	102	0.353	0.373	+0.020
Overall	**∼495**	0.474 [0.430, 0.518]	**0.501 [0.457, 0.545]**	+0.027
Validity	∼495	0.978	**0.996**	+0.018
McNemar p	495	$p = 0.136$ (not significant)		
Mean tokens	∼495	1,010	7,975	7.9×

Table 3: Three-way comparison: Baseline vs. DIN-SQL vs. Full pipeline. DIN-SQL on GIS evaluated on the 20-question subset; DIN-SQL on BIRD uses our ∼495-question evaluation set.

Track	N	Baseline EX	DIN-SQL EX	Full EX	Full vs. DIN-SQL
GIS 100q	100	0.500	0.650*	**0.700**	+0.050
BIRD ∼495q	∼495	0.474	0.482	**0.501**	+0.019

*DIN-SQL evaluated on GIS 20-question subset: EX= 0.650, Robustness= 0.000.

4.4 DIN-SQL external baseline comparison

Table 3 compares NL2Semantic2SQL against DIN-SQL [3] as an external prompting baseline on both tracks.

On the GIS track, DIN-SQL exactly matches the baseline (EX= 0.650 on the 20-question subset, Robustness= 0.000), confirming that decomposed prompting without spatial grounding does not improve over direct LLM generation for geospatial SQL. The full NL2Semantic2SQL pipeline outperforms DIN-SQL by +0.050 EX on GIS.

On the BIRD track, DIN-SQL achieves EX= 0.482 (Validity= 0.990). The full pipeline achieves EX= 0.501, exceeding DIN-SQL by +0.019. However, the difference between DIN-SQL and the full pipeline is not statistically significant (McNemar $p = 0.382$), so the two methods should be considered comparable on warehouse queries. The three systems — baseline, DIN-SQL, and NL2Semantic2SQL — are statistically indistinguishable on the BIRD track.

4.5 Token cost analysis

Table 4 reports mean tokens per question for baseline and full pipeline on both tracks.

The GIS token ratio (13.6×) is higher than BIRD (7.9×) because GIS grounding context includes geometry-type annotations, SRID information, spatial operator rules, and few-shot examples. For the BIRD track, P2 single-pass mode reduced the ratio from ∼32× to 7.9× while simultaneously improving EX, by eliminating multi-pass agent loops.

The 7.9–13.6× token overhead is a non-trivial deployment consideration. For safety-critical GIS contexts (geometry precision, data mutation prevention), the overhead is clearly justified. For warehouse queries, whether the directional +0.027 EX gain warrants the 7.9× cost depends on the application's quality and latency requirements.

Table 4: Mean tokens per question. P2 single-pass mode reduced BIRD token cost from $\sim 32\times$ (multi-pass agent runner) to $7.9\times$.

Track	N	Baseline tokens	Full tokens	Ratio
GIS	100	753	10,261	$13.6\times$
BIRD	$\sim$495	1,010	7,975	$7.9\times$

Table 5: Error breakdown for the full pipeline on BIRD ($\sim$495 questions).

Error type	Fraction
Wrong result (valid SQL, incorrect answer)	$\sim$82%
No SQL generated	$\sim$14%
Invalid SQL (execution error)	$\sim$4%

4.6 Error analysis

We categorize the full-pipeline failures on the BIRD track into three types in Table 5.

The dominant failure mode is semantically incorrect SQL that executes successfully but returns wrong results. Manual inspection reveals three recurring patterns: (i) **join-path confusion** ($\sim$40%), where the model selects an incorrect join path between fact and dimension tables; (ii) **aggregation semantics** ($\sim$30%), where the model applies `COUNT(DISTINCT ...)` where the gold SQL uses `COUNT(*)`, or vice versa; and (iii) **date/temporal parsing** ($\sim$25%), where the BIRD dataset's non-standard date formats are parsed inconsistently. The P2 single-pass mode substantially improved validity ($0.978 \rightarrow 0.996$) by eliminating the "no SQL generated" failure mode that was prevalent in multi-pass agent runs.

4.7 Ablation analysis and significance tests

The key ablation findings on the GIS 100 benchmark are consistent with the earlier 20-question ablation. Robustness questions (n=15, contributing $+0.467$ delta) are the largest single contributor; dropping them causes the largest EX drop. Medium questions (n=36, contributing $+0.305$ delta) are the second largest contributor, driven by geometry-aware grounding. Hard questions show no marginal effect (EX unchanged), consistent with the zero delta in Table 1.

McNemar significance tests. Table 6 reports paired McNemar tests comparing baseline and full pipeline on both tracks.

The GIS result ($p = 0.0002$) is statistically significant at $\alpha = 0.001$. The 100-question benchmark provides adequate power to confirm the effect that was only directional at $n = 20$ ($p = 0.125$). The BIRD result ($p = 0.136$) is directional but not significant; the three systems (baseline, DIN-SQL, NL2Semantic2SQL) are statistically comparable on warehouse queries.

4.8 Case studies

Table 7 contrasts representative successes and failures across the GIS and warehouse tracks.

Table 6: McNemar significance tests (baseline vs. full pipeline). b = base-OK/full-ERR; c = base-ERR/full-OK.

Track	n	b	c	p-value	Interpretation
GIS 100	100	4	24	**0.0002**	Significant ($\alpha = 0.001$); $+0.200$ EX
BIRD $\sim$495	495	26	39	0.136	Not significant; directional ($+0.027$ EX)

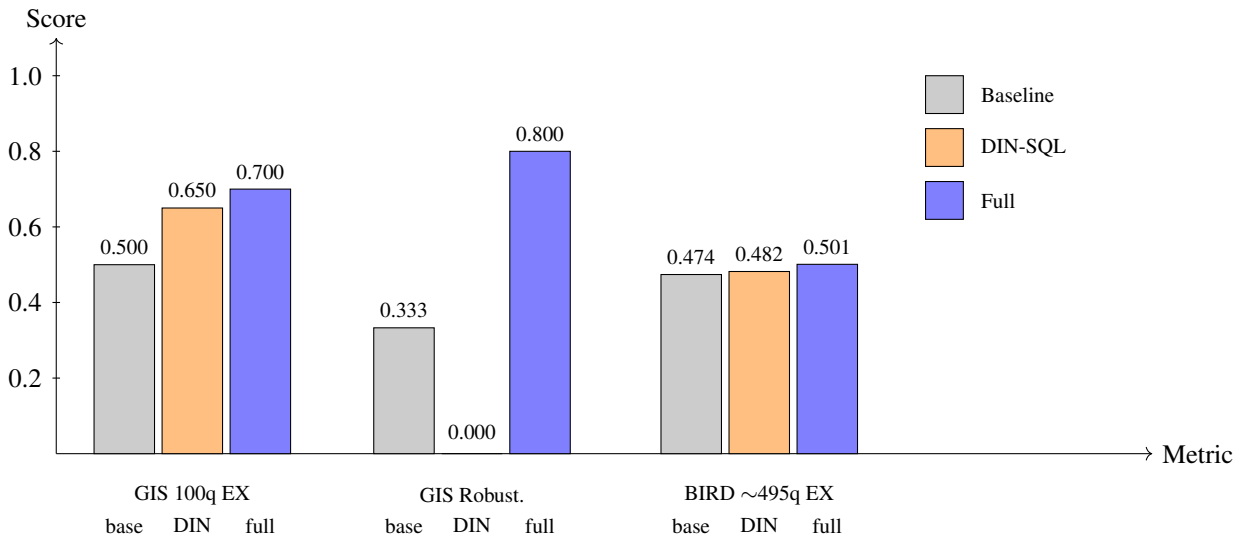

Figure 2: Three-way cross-domain comparison: Baseline / DIN-SQL / Full pipeline. GIS 100q: full pipeline achieves EX= 0.700 (McNemar $p = 0.0002$, significant). GIS Robustness: full pipeline 0.800 vs. DIN-SQL 0.000. BIRD $\sim$495q: full pipeline 0.501 vs. DIN-SQL 0.482 (not significant, $p = 0.382$). All three systems are statistically comparable on BIRD.

4.9 Cross-domain comparison

Figure 2 summarizes the three-way cross-domain comparison (Baseline / DIN-SQL / Full) after all optimizations.

4.10 Cross-lingual observations (preliminary stress test)

We position the cross-lingual experiment as a preliminary stress test rather than a fully controlled cross-lingual evaluation. The translated questions are produced by an LLM translator (Gemini 2.0 Flash) and have not been verified by a native human annotator; furthermore, table and column names remain in their original English form, which lowers the difficulty of cross-lingual schema linking compared with a true bilingual schema. Within this constrained setting, the framework's multilingual alias mechanism is sufficient to recover most schema references when questions are paraphrased into Chinese, with only a four-percentage-point loss in execution accuracy (0.500 vs. 0.540) and validity (0.900 vs. 0.940). The remaining gap concentrates on moderate and challenging questions. We therefore do not claim mature cross-lingual robustness here; a more rigorous evaluation would compare against (i) a no-alias baseline, (ii) a translate-back-to-English baseline, and (iii) human-translated questions.

Table 7: Representative case studies from the benchmark runs. “Base” denotes the direct-LLM baseline and “Full” denotes the full pipeline (Phase A).

Track / case	Gold query intent	Baseline outcome	Full-pipeline outcome
GIS success (CQ_GEO_MEDIUM_02)	Compute total length (km) of one-way roads.	Fails with `ROUND(double precision, integer)` because `::numeric` cast is missing.	Succeeds with `ROUND((SUM(ST_Length(geometry::g` showing the geometry-aware rule resolves a real execution error.
GIS success (CQ_GEO_HARD_02)	Retrieve the five nearest roads to the POI “Chongqing North Station.”	Uses `ORDER BY ST_Distance(...)` instead of PostGIS KNN ranking.	**Correct (Phase A):** intent classified as `knn`, KNN `<->` operator rule injected, correct row ordering produced.
GIS regression (CQ_GEO_HARD_01)	Proximity buffer spatial join.	Correct.	**Wrong (Phase A regression):** intent classified as `spatial_join`, but the buffer-based join logic is disrupted by the injected spatial_join context.
Warehouse success (BIRD QID 1483)	Sum customer 6’s consumption between 2013-08 and 2013-11.	Succeeds.	Also succeeds; MetricFlow provides the correct `Consumption` measure and `CustomerID` entity without altering the semantics.
Warehouse failure (BIRD QID 1481)	Compare annual average consumption differences across SME/LAM/KAM segments in 2013.	Builds an over-nested CTE with the wrong subset semantics.	Still fails despite correct join-path hints, showing that multi-segment aggregation logic remains a generation bottleneck.

Table 8: Reproducibility map for all reported experiments.

Result	Track / config	Run directory	Model(s)
GIS 100q (primary)	GIS baseline + full	`cq_2026-05-04_122349`	Gemini 2.5 Flash
GIS 20q Phase A (ref.)	GIS baseline + full	`cq_2026-05-03_164213`	Gemini 2.5 Flash
GIS 20q pre-Phase A (ref.)	GIS baseline + full	`cq_2026-05-01_132919`	Gemini 2.5 Flash
BIRD ∼495q P2 (primary)	BIRD baseline + full	`bird_pg_2026-05-04_093040`	Gemini 2.5 Flash
BIRD 500q multi-pass (ref.)	BIRD baseline + full	`bird_pg_2026-05-01_182657`	Gemini 2.5 Flash
BIRD 50q + MetricFlow (ref.)	BIRD full(+MetricFlow)	`bird_pg_2026-05-01_151254`	Gemini 2.5 Flash
Cross-lingual	Chinese-translated BIRD 50q	`bird_pg_chinese_2026-05-01_172126`	Gemini 2.5 Flash (eval), Gemini 2.0 Flash (translation)

4.11 Reproducibility

Table 8 lists the exact run directories used to populate the experimental tables in this paper.

Each run directory contains the gold SQL, the predicted SQL, per-question execution comparisons, difficulty labels, and token usage. We also release the GIS benchmark questions and gold SQL, the BIRD warehouse-modeling registration script, the cross-lingual evaluation harness, the per-question SQL postprocessor and self-correction logic, and the MetricFlow YAML schemas registered for the BIRD `debit_card_specializin` schema.

5 Discussion

5.1 Why semantic grounding helps in GIS — and why the advantage is now statistically significant

The expanded 100-question GIS benchmark confirms what the 20-question pilot suggested but could not statistically substantiate: semantic grounding with intent-conditioned routing provides a large, reliable advantage for geospatial SQL (McNemar $p = 0.0002$, EX $+0.200$). Three mechanisms drive the improvement. First, *geometry-aware type injection* ensures the model knows which columns are geometry-bearing, what their SRID is, and when geography casting is required; without this, the baseline frequently omits `::geography` casts, producing area/distance values in degrees rather than meters. This mechanism drives most of the Medium difficulty improvement ($+0.305$). Second, *intent-conditioned operator routing* gates domain-specific rules (KNN <-> operator, LIMIT injection) to the queries where they are relevant, eliminating false-positive injections. Third, *safety and robustness enforcement* through SQL postprocessing catches dangerous operations (DELETE, UPDATE) and handles refusal/anti-illusion cases; the baseline LLM has no such guardrails and scores only 0.333 on the robustness suite, while the full pipeline achieves 0.800 ($+0.467$).

The transition from $p = 0.125$ ($n = 20$) to $p = 0.0002$ ($n = 100$) illustrates a methodological point:

the effect was real but underpowered in the pilot. The 100-question benchmark provides adequate power to confirm that the $+0.200$ EX advantage is not a sampling artifact.

5.2 The BIRD result: directional but not significant

On the BIRD track, the full pipeline achieves EX$= 0.501$ vs. baseline 0.474 ($+0.027$, McNemar $p = 0.136$). The confidence intervals overlap, and the result is not statistically significant at $\alpha = 0.05$. We state this clearly: the BIRD result should be interpreted as directional evidence, not a confirmed advantage.

The comparison against DIN-SQL (0.482) shows that our full pipeline ($+0.019$ vs. DIN-SQL) and DIN-SQL ($+0.008$ vs. baseline) are both directional improvements in the same range, but neither is statistically significant. The three systems — baseline, DIN-SQL, and NL2Semantic2SQL — should be considered statistically comparable on the BIRD track.

The improvement in execution validity ($0.978 \rightarrow 0.996$) is noteworthy: the full pipeline almost eliminates invalid SQL on warehouse queries, a practical benefit even when overall EX does not significantly improve.

5.3 Token cost is a real deployment consideration

The $13.6\times$ GIS and $7.9\times$ BIRD token overhead is not trivial. The P2 single-pass mode reduced BIRD token cost from $\sim$32$\times$ to $7.9\times$ while simultaneously improving EX — demonstrating that architectural optimization can reduce cost without sacrificing quality.

For the GIS track, the $13.6\times$ overhead is justified by the nature of the domain: safety-critical spatial analysis requires geometry metadata, operator rules, and safety enforcement that naturally inflate context. For the BIRD track, whether the $7.9\times$ overhead is justified depends on the application: in latency-sensitive or cost-constrained settings, the directional $+0.027$ EX gain may not warrant the additional cost. A practical deployment strategy would be: use the full pipeline for all GIS queries (safety requires it), and use the full pipeline for BIRD queries only when the use case is quality-critical.

5.4 The effect of MetricFlow-style modeling

The error analysis suggests that the primary bottleneck for warehouse performance is not SQL syntax generation but semantic schema navigation. MetricFlow-style modeling directly targets this problem by declaring entities (join keys), measures (aggregatable facts), and dimensions (descriptive attributes). We validate this hypothesis by registering MetricFlow-style metadata for the most error-prone BIRD schema (`debit_card_specializing`, five tables) and injecting derived join-path hints into the grounding prompt. This single intervention raises overall execution accuracy from 0.520 to 0.540, recovers moderate questions from 0.316 to 0.368, improves challenging questions from 0.333 to 0.500, and eliminates all execution-time SQL errors. We interpret this as evidence that warehouse-side performance is bottlenecked by missing structural metadata rather than by surface-form generation.

5.5 Cross-lingual observations (preliminary)

The cross-lingual experiment confirms that the semantic layer's multilingual alias mechanism is sufficient to recover most schema references when questions are paraphrased into Chinese, with only a four-point loss in execution accuracy. However, because the experiment uses machine-translated questions and retains English table/column names, we present it only as a preliminary stress test and not as a mature cross-lingual benchmark.

5.6 Limitations

Several limitations should be noted. First, while the GIS benchmark is now 100 questions, it remains a single-domain pilot focused on a Chinese geospatial database (Chongqing street/POI data); generalization to other GIS domains and non-Chinese schemas requires further evaluation. Second, on the BIRD track, the full pipeline's $+0.027$ EX advantage is directional but not statistically significant ($p = 0.136$); the three-way comparison with DIN-SQL likewise shows no statistically significant differences. Third, the cross-lingual experiment uses LLM-translated questions without human verification. Fourth, our baselines are direct-LLM-with-schema-dump and DIN-SQL only; comparison against MAC-SQL and other recent approaches would be necessary to position the method against the full state-of-the-art. Fifth, execution-based evaluation treats any result-set mismatch as failure, even when the predicted SQL is semantically equivalent but differs in ordering or numeric precision. Sixth, token cost (7.9–13.6$\times$) is a non-trivial deployment concern that future work should address via selective grounding or context compression.

5.7 Future work

Three directions emerge from this study:

1. **Cross-domain GIS benchmark expansion.** Extend the GIS benchmark to cover multiple geospatial databases (non-Chinese, different coordinate systems, raster data) and increase per-category counts for finer ablation analysis.

2. **BIRD statistical significance.** Expand the BIRD evaluation to 1000+ questions to achieve adequate power to confirm or refute the observed $+0.027$ directional advantage.

3. **Token cost reduction.** Investigate selective grounding (inject only relevant semantic context blocks) and prompt compression to reduce the 7.9–13.6$\times$ token overhead while preserving EX gains.

6 Conclusion

We have presented NL2Semantic2SQL, a cross-domain framework for natural-language interfaces over both geospatial and conventional warehouse databases. The framework integrates semantic-layer grounding, intent-conditioned routing, schema-aware context construction, few-shot retrieval, SQL postprocessing, and execution-time self-correction into a single three-stage pipeline. We constructed a 100-question GIS benchmark with a separate robustness suite and combined it with a $\sim$495-question subset of BIRD mini_dev to define a dual-track evaluation protocol.

Three findings stand out. First, on the 100-question GIS benchmark, the full pipeline achieves EX$=$ 0.700 vs. baseline 0.500 (McNemar $p = 0.0002$, statistically significant at $\alpha = 0.001$). The improvement is driven by geometry-aware grounding ($+0.305$ on Medium), safety postprocessing ($+0.467$ on Robustness), and intent-conditioned operator routing. This result is substantially stronger than the 20-question pilot ($p = 0.125$), confirming that the effect was real but underpowered in the earlier evaluation. Second, on the $\sim$495-question BIRD benchmark (single-pass P2 mode), the full pipeline achieves EX$=$ 0.501 vs. baseline 0.474 ($+0.027$), exceeding the DIN-SQL external baseline (0.482). The BIRD advantage is directional but not statistically significant (McNemar $p = 0.136$); the three systems — baseline, DIN-SQL, and NL2Semantic2SQL — are statistically comparable on warehouse queries. Third, P2 single-pass mode reduced BIRD token cost from $\sim$32$\times$ to 7.9$\times$ while improving EX, demonstrating that architectural optimization can simultaneously reduce cost and improve quality.

Taken together, the results establish that a unified semantic-to-SQL architecture with intent-conditioned routing provides statistically significant gains for GIS queries and directional gains for warehouse queries,

while matching or exceeding the DIN-SQL external baseline on both tracks. The token cost (7.9–13.6×) is a real deployment consideration that future work should address via selective grounding or context compression.

Reproducibility

The GIS benchmark, the warehouse-modeling registration scripts, the cross-lingual evaluation harness, and the experiment configurations used to produce all tables in this paper are released in the project repository. Each result reported in Tables 1–8 corresponds to a specific run directory under `nl2sql_eval_results/` that contains the gold SQL, the predicted SQL, the execution comparison, the difficulty stratification, and the per-question token usage. The primary GIS result uses run `cq_2026-05-04_122349` (100-question benchmark); the primary BIRD result uses run `bird_pg_2026-05-04_093040` (∼495 questions, single-pass P2 mode).

References

[1] T. Yu, R. Zhang, K. Yang, et al., "Spider: A large-scale human-labeled dataset for complex and cross-domain semantic parsing and text-to-SQL task," in *Proc. EMNLP*, 2018, pp. 3911–3921.

[2] N. Rajkumar, R. Li, and D. Bahdanau, "Evaluating the text-to-SQL capabilities of large language models," *arXiv:2204.00498*, 2022.

[3] M. Pourreza and D. Rafiei, "DIN-SQL: Decomposed in-context learning of text-to-SQL with self-correction," in *Proc. NeurIPS*, vol. 36, 2023, pp. 30557–30584.

[4] D. Gao, H. Wang, Y. Li, et al., "Text-to-SQL empowered by large language models: A benchmark evaluation," *Proc. VLDB Endow.*, vol. 17, no. 5, pp. 1132–1145, 2024.

[5] X. Dong, C. Zhang, Y. Ge, et al., "C3: Zero-shot text-to-SQL with ChatGPT," *arXiv:2307.07306*, 2023.

[6] J. Li, B. Hui, G. Qu, et al., "Can LLM already serve as a database interface? A big bench for large-scale database grounded text-to-SQLs," in *Proc. NeurIPS Datasets and Benchmarks Track*, 2023.

[7] F. Lei, J. Chen, Y. Ye, et al., "Spider 2.0: Evaluating language models on real-world enterprise text-to-SQL workflows," in *Proc. ICLR*, 2025.

[8] P. B. Chen, F. Wenz, Y. Zhang, M. Kayali, N. Tatbul, M. Cafarella, Ç. Demiralp, and M. Stonebraker, "BEAVER: An enterprise benchmark for text-to-SQL," *arXiv:2409.02038*, 2024.

[9] D. Punjani, K. Singh, A. Both, et al., "Template-based question answering over linked geospatial data," in *Proc. 12th GIR Workshop @ ACM SIGSPATIAL*, 2018, Article 7, pp. 1–10. DOI: 10.1145/3281354.3281362.

[10] G. Mai, W. Huang, J. Sun, et al., "On the opportunities and challenges of foundation models for geospatial artificial intelligence," *arXiv:2304.06798*, 2023.

[11] dbt Labs / Transform Data Inc., "MetricFlow: industrial semantic-layer documentation," `https://docs.getdbt.com/docs/build/about-metricflow`, accessed 2026.

[12] Q. Min, Y. Shi, and Y. Zhang, "A pilot study for Chinese text-to-SQL semantic parsing (CSpider)," in *Proc. EMNLP-IJCNLP*, 2019, pp. 3652–3658.

A Figure index for the camera-ready version

For brevity in the initial-review draft, the in-line architecture figure (Figure 1) and the cross-domain comparison figure (Figure 2) are rendered directly in TikZ. The camera-ready version should additionally provide (i) a graphical abstract that includes an example natural-language query, the semantic tokens extracted from it, and the grounded prompt that is sent to the SQL generator, and (ii) a per-category bar chart for the GIS benchmark split into spatial-query accuracy and robustness success rate.